

Paquetes y módulos

Paquetes

Un **paquete** es una colección de archivos que contiene código fuente, definiciones de constantes, definiciones de tipos, etc. Todos los archivos de un paquete se encuentran en un mismo directorio.

Los paquetes organizan el código en forma lógica. Las variables, funciones y tipos de datos definidos dentro de un paquete son privados del mismo, es decir, no se pueden utilizar afuera, a menos que se exporten explícitamente.

Además de que todos los archivos estén en el mismo directorio, cada archivo que forma parte de un paquete debe declarar a qué paquete pertenece:

```
package mipaquete
```

Módulos

A su vez una colección de paquetes pueden conformar un **módulo**. Un módulo es la unidad fundamental de organización y distribución de código. Los módulos permiten gestionar dependencias, compartir código y versionarlo.

Un módulo se define en una carpeta que contiene un archivo `go.mod`. En este archivo se especifica el nombre del módulo, la versión de Go con la que se desarrolló y las dependencias de otros módulos.

Todos los paquetes (en general cada paquete en una subcarpeta) que están en la misma carpeta que contiene el archivo `go.mod` son parte del mismo módulo. En general un módulo corresponde a un proyecto.

Gestión de módulos

Para crear un módulo se usa el comando `go mod init` dentro de la carpeta que contendrá el proyecto, seguido del nombre del módulo. Usualmente el nombre del módulo es el repositorio donde se aloja. Ejemplo:

```
go mod init github.com/untref-ayp2/miproyecto
```

Este comando creará un nuevo archivo llamado `go.mod` con el siguiente contenido:

```
module github.com/untref-ayp2/miproyecto
```

```
go 1.24.1
```

Todos los archivos que se encuentren dentro de la misma carpeta en la que se haya creado `go.mod` son los que serán considerados parte del módulo `github.com/untref-ayp2/miproyecto`

```
miproyecto
├── otra paquete
│   ├── estructuras.go
│   └── estructuras_test.go
├── un paquete
│   ├── utils.go
│   └── utils_test.go
├── go.mod
├── go.sum
└── main.go
```

Importar módulos/paquetes

El objetivo final de organizar nuestro código en paquetes y módulos es que podamos reusarlos en otros proyectos. Cada vez que necesitemos usar una dependencia tanto interna a nuestro módulo

como externa, debemos usar la instrucción `import`, lo que nos permite acceder a las estructuras, funciones y utilidades de los distintos módulos.

En Go existen distintos tipos de módulos que podemos importar, dependiendo de su procedencia. Por ejemplo, Go provee una serie de módulos que son nativos del lenguaje, como pueden ser:

fmt entrada y salida por pantalla y teclado
math constantes y funciones matemáticas
time funciones para medir y mostrar tiempos
errors funciones para manipular errores
path funciones para manipular carpetas y archivos
testing soporte para test automáticos
strings funciones para manipular cadenas de caracteres

Hasta ahora hemos utilizado mayormente este tipo de módulos. También podemos utilizar referencias a paquetes dentro de nuestro mismo módulo.

```
package main

import "github.com/untref-ayp2/miproyecto/saludo"

func main() {
    saludo.Saludar()
}
```

La ruta completa del import (`github.com/untref-ayp2/miproyecto/saludo`) indica que debemos importar el paquete `saludo`, que es un subpaquete de nuestro módulo. El nombre del paquete (`saludo`) es la última parte de la ruta y es la referencia que usamos para acceder a sus miembros exportados.

Otros tipos de módulos que podemos importar son los módulos de otros programadores, por ejemplo, podríamos pensar en utilizar un supuesto módulo de algoritmos de ordenamiento de la cátedra. Entonces, imaginemos que existe el módulo llamado `"untref.edu.ar/ayp2/busqueda"`. Podemos importarlo de la siguiente manera.

```
package main

import (
    "fmt"
    "untref.edu.ar/ayp2/busqueda"
)

func main() {
    lista := []int{2, 3, 9, 5, 1, 8, 4, -5, -4, 6}
    busqueda.Quicksort(lista)
    fmt.Println(lista)
}
```

El resultado será:

```
[-5 -4 1 2 3 4 5 6 8 9]
```

Hay varios puntos a notar:

- Siempre se importan **paquetes**.

- Cuando importamos un paquete, el texto luego de la última / será el nombre de la referencia que podemos utilizar para acceder a los miembros del paquete, que además coincide con el nombre del paquete.
- Podemos renombrar la referencia que se crea al importar un paquete, ya sea por conveniencia como en el caso de que haya un conflicto de nombres entre paquetes de distintos módulos.

```
import b "untref.edu.ar/ayp2/busqueda"
```

Luego podremos utilizar la nueva referencia b en lugar de busqueda.

Dependencias

Cuando necesitamos utilizar un módulo de terceros, como podría ser un módulo que provee distintos métodos de ordenamiento, primero debemos encontrar el módulo, esto puede ser haciendo una búsqueda en el registro de go que podemos encontrar en <https://pkg.go.dev/> o bien, buscando módulos en internet. Es posible instalar módulos directamente desde repositorios de Git.

Para instalar y agregar dicha dependencia en nuestro módulo, podemos utilizar la herramienta go get de la siguiente forma:

```
go get github.com/untref-ayp2/busquedas
```

Una vez ejecutado este comando, el archivo go.mod, donde se encuentra la definición de nuestro módulo, se verá similar a:

```
module github.com/untref-ayp2/miproyecto
```

```
go 1.24.1
```

```
require github.com/untref-ayp2/busquedas v0.1.0
```

De esta forma, si estamos trabajando con más personas en el mismo proyecto, o si alguien más quiere utilizar nuestro módulo como dependencia en su propio proyecto, será posible respetar los requerimientos que cada módulo en el proyecto necesita que se cumplan.

Si en lugar de instalar y agregar la dependencia con go get, tenemos una forma alternativa que es, primero usar la dependencia en nuestro código y luego ejecutar go mod tidy. Veamos un ejemplo.

Supongamos que creamos un módulo e importamos algunas cosas en nuestro archivo main.go.

Ejemplo de creación de un módulo simple

Primero creamos una carpeta con el nombre mimodulo y nos posicionamos en ella.

```
mkdir mimodulo && cd mimodulo
```

Inicializamos el módulo.

```
go mod init github.com/untref-ayp2/mimodulo
```

```
go: creating new go.mod: module github.com/untref-ayp2/mimodulo
```

```
cat go.mod
```

```
module github.com/untref-ayp2/mimodulo
```

```
go 1.24.1
```

Usando cualquier método que les resulte conveniente creamos el archivo main.go con el siguiente código.

```
cat main.go
```

```

package main

import (
    "fmt"
    "github.com/fatih/color"
)

func main() {
    msj := color.RedString(";Texto en rojo!")
    fmt.Println(msj)
}

```

Ejecutamos nuestro programa, pero vemos que hay un error.

```
go run main.go
```

```

main.go:5:2: no required module provides package github.com/fatih/color; to add it:
    go get github.com/fatih/color

```

Podemos notar que los mensajes de error de Go muchas veces (no siempre) son muy descriptivos y hasta nos pueden sugerir la solución si prestamos especial atención a lo que nos dice.

Si bien en este ejemplo, queremos mostrar una forma alternativa de solucionar la falta de la dependencia, `go get` es otra forma de arreglar el problema que, intencionalmente, acabamos de crear.

Para ello usaremos `go mod tidy`, si consultamos la ayuda podemos aprender qué es lo que este comando realmente hace con nuestro módulo.

```
go help mod tidy
```

```
[...]
```

```

Tidy makes sure go.mod matches the source code in the module.
It adds any missing modules necessary to build the current module's
packages and dependencies, and it removes unused modules that
don't provide any relevant packages. It also adds any missing entries
to go.sum and removes any unnecessary ones.

```

```
[...]
```

Entonces, ya sabemos que `go mod tidy` se va a ocupar de agregar o corregir cualquier tipo de dependencia perdida en nuestro código.

```
go mod tidy
```

```

go: finding module for package github.com/fatih/color
go: downloading github.com/fatih/color v1.18.0
go: found github.com/fatih/color in github.com/fatih/color v1.18.0
go: downloading golang.org/x/sys v0.25.0
go: downloading github.com/mattn/go-colorable v0.1.13

```

Si ahora vemos cómo luce el archivo `go.mod` podemos comprobar que se agregó la directiva `require` donde se declaran las dependencias faltantes (pero adicionalmente `go mod tidy` también descargó esas dependencias en nuestro proyecto).

```
cat go.mod
```

```
module github.com/untref-ayp2/mimodulo
```

```
go 1.24.1
```

```
require github.com/fatih/color v1.18.0
```

```
require (  
    github.com/mattn/go-colorable v0.1.13 // indirect  
    github.com/mattn/go-isatty v0.0.20 // indirect  
    golang.org/x/sys v0.25.0 // indirect  
)
```

Si ahora ejecutamos nuevamente nuestro programa, todo debería funcionar como se espera.

```
go run main.go
```

¡Texto en rojo!

(el mensaje se imprime en color rojo en la terminal)

¡Para practicar!

Se recomienda recrear el ejemplo previamente presentado y ejecutarlo para corroborar que todo funciona correctamente.

Los ejemplos de este capítulo están en 02-paquetes-y-modulos/ejemplos/ del repositorio taller-go.

Ejercicios

Los ejercicios de este capítulo están en 02-paquetes-y-modulos/ejercicios/ del repositorio taller-go.